

Wingman

QueryBuilder & QueryableModel

Developer Reference

Overview

Wingman provides two closely related classes for database interaction:

QueryBuilder — A low-level, fluent SQL builder. Accepts a mysqli connection on every terminal method call.

QueryableModel — A high-level model wrapper around QueryBuilder. Manages the connection and state internally, exposing the same chainable API without needing to pass a connection every call.

In most cases you will use QueryableModel. QueryBuilder is available if you need raw query construction outside of a model context.

Selecting Data

select()

Specify which columns to return. Accepts one or more column names. Defaults to SELECT * if omitted.

```
// Select specific columns
$users = $user->select('id', 'name', 'email')->get();

// Select with table prefix (useful for joins)
$posts = $post->select('posts.id', 'posts.title', 'users.name')->get();
```

where()

Adds a WHERE condition. Multiple where() calls are joined with AND.

```
// Simple equality
$user = $user->where('id', '=', 1)->first();

// Comparison operators
```

```
$adults = $user->where('age', '>=', 18)->get();

// Multiple conditions (AND)
$result = $user->where('active', '=', 1)
    ->where('role', '=', 'admin')
    ->get();
```

orWhere()

Adds an OR WHERE condition.

```
$result = $user->where('role', '=', 'admin')
    ->orWhere('role', '=', 'moderator')
    ->get();
```

whereNull() / whereNotNull()

Checks whether a column is NULL or NOT NULL.

```
// Find users without a profile picture
$users = $user->whereNull('avatar')->get();

// Find users with a verified email
$users = $user->whereNotNull('email_verified_at')->get();
```

whereIn() / whereNotIn()

Filters rows where a column value is inside or outside a given array.

```
// Users in specific roles
$users = $user->whereIn('role', ['admin', 'moderator', 'editor']->get();

// Posts not in these categories
$posts = $post->whereNotIn('category', ['draft', 'archived']->get();
```

whereBetween()

Filters rows where a column falls within a range (inclusive).

```
// Posts created in January 2025
$posts = $post->whereBetween('created_at', '2025-01-01', '2025-01-31')->get();

// Products in price range
$items = $product->whereBetween('price', 100, 500)->get();
```

whereLike()

Filters rows using a LIKE pattern. Use % as a wildcard.

```
// Search by name
$users = $user->whereLike('name', '%john%')->get();
```

```
// Find emails from a specific domain
$users = $user->whereLike('email', '%@gmail.com')->get();
```

Joining Tables

All join methods accept the joined table name, the two columns to compare, and the operator.

join() — INNER JOIN

```
$posts = $post->select('posts.title', 'users.name')
    ->join('users', 'posts.user_id', '=', 'users.id')
    ->get();
```

leftJoin()

```
// All posts, even those without a category
$posts = $post->leftJoin('categories', 'posts.category_id', '=', 'categories.id')
    ->get();
```

rightJoin()

```
// All categories, even those without posts
$data = $post->rightJoin('categories', 'posts.category_id', '=', 'categories.id')
    ->get();
```

crossJoin()

```
// Cartesian product of two tables
$data = $product->crossJoin('colors')->get();
```

Ordering & Grouping

orderBy() / orderByDesc()

```
// Ascending order (default)
$users = $user->orderBy('name')->get();

// Descending order
$posts = $post->orderByDesc('created_at')->get();

// Multiple order clauses
$users = $user->orderBy('last_name')
    ->orderBy('first_name')
    ->get();
```

groupBy()

```
// Count posts per user
$counters = $post->select('user_id', 'COUNT(*) as total')
    ->groupBy('user_id')
    ->get();
```

having()

Filters grouped results. Must be used after groupBy().

```
// Users with more than 5 posts
$users = $post->select('user_id', 'COUNT(*) as total')
    ->groupBy('user_id')
    ->having('total', '>', 5)
    ->get();
```

Limiting Results

limit() / offset()

```
// Get only 10 results
$users = $user->limit(10)->get();

// Skip first 20, get next 10
$users = $user->limit(10)->offset(20)->get();
```

paginate()

A convenience method that calculates limit and offset from a page number and page size.

```
// Page 2, 15 items per page
$users = $user->paginate(2, 15)->get();
```

Terminal Methods

Terminal methods execute the query and return results. After calling one, the query state is automatically reset.

get()

Returns all matching rows as an array of associative arrays. Accepts an optional array of column keys to exclude from the result.

```
// All users
$users = $user->get();

// Exclude sensitive fields
$users = $user->get(['password', 'remember_token']);
```

first()

Returns the first matching row as an associative array, or null if no match is found.

```
// First active user
$user = $user->where('active', '=', 1)->first();

// Exclude password from result
$user = $user->where('id', '=', $id)->first(['password']);
```

find()

Finds a single record by its primary key. Only available on QueryableModel.

```
// Find by default primary key (id)
$user = $user->find(42);

// Find by custom primary key
$product = $product->find('SKU-001', 'sku');

// Find and exclude fields
$user = $user->find(42, 'id', ['password']);
```

all()

Returns every row in the table without any conditions. Only available on QueryableModel.

```
$users = $user->all();

// Exclude fields from all rows
$users = $user->all(['password']);
```

count()

Returns the total number of rows matching the current query.

```
// Total users
$total = $user->count();

// Count only active users
$active = $user->where('active', '=', 1)->count();
```

paginateResult()

Executes the query with pagination and returns a structured result containing the data and pagination metadata.

```
$result = $post->where('active', '=', 1)
    ->orderByDesc('created_at')
    ->paginateResult(1, 10);
```

```
// Returns:  
// [  
//   'data'      => [...],  
//   'total'     => 100,  
//   'per_page'  => 10,  
//   'current_page' => 1,  
//   'last_page' => 10,  
//   'from'      => 1,  
//   'to'        => 10,  
// ]
```

Write Operations

insert()

Inserts a single row and returns the inserted record's ID.

```
$id = $user->insert([  
  'name'  => 'John Doe',  
  'email' => 'john@example.com',  
  'role'  => 'user',  
]);
```

insertMany()

Inserts multiple rows in a single query and returns the number of rows affected.

```
$count = $user->insertMany([  
  ['name' => 'Alice', 'email' => 'alice@example.com'],  
  ['name' => 'Bob', 'email' => 'bob@example.com'],  
]);
```

update()

Updates matching rows with the given data. Always chain a where() condition to avoid updating the entire table.

```
$affected = $user->where('id', '=', 42)  
  ->update([  
    'name'      => 'Jane Doe',  
    'updated_at' => '2025-04-30 12:00:00',  
  ]);
```

delete()

Deletes matching rows and returns the number of rows affected. Always chain a where() condition.

```
$deleted = $user->where('id', '=', 42)->delete();
```

Calling delete() without a where() condition will delete all rows in the table.

Chaining Examples

All methods are fully chainable. Below are real-world examples combining multiple clauses.

Complex Read Query

```
$results = $post->select('posts.id', 'posts.title', 'users.name',  
    'categories.label')  
    ->join('users', 'posts.user_id', '=', 'users.id')  
    ->leftJoin('categories', 'posts.category_id', '=', 'categories.id')  
    ->where('posts.active', '=', 1)  
    ->whereIn('posts.category_id', [1, 2, 3])  
    ->orderByDesc('posts.created_at')  
    ->paginateResult(1, 10);
```

Search Query

```
$users = $user->whereLike('name', '%john%')  
    ->orWhere('email', 'LIKE', '%john%')  
    ->where('active', '=', 1)  
    ->orderBy('name')  
    ->get(['password']);
```

Aggregation Query

```
$stats = $order->select('user_id', 'COUNT(*) as total_orders', 'SUM(amount) as  
    total_spent')  
    ->where('status', '=', 'completed')  
    ->groupBy('user_id')  
    ->having('total_orders', '>', 5)  
    ->orderByDesc('total_spent')  
    ->get();
```

Quick Reference

Method	Description
<code>select(...\$cols)</code>	Choose which columns to return
<code>where(\$col, \$op, \$val)</code>	Add an AND WHERE condition
<code>orWhere(\$col, \$op, \$val)</code>	Add an OR WHERE condition
<code>whereNull(\$col)</code>	Match rows where column IS NULL
<code>whereNotNull(\$col)</code>	Match rows where column IS NOT NULL
<code>whereIn(\$col, \$values)</code>	Match rows where column is in array
<code>whereNotIn(\$col, \$values)</code>	Match rows where column is not in array
<code>whereBetween(\$col, \$min, \$max)</code>	Match rows where column is in range

Method	Description
<code>whereLike(\$col, \$pattern)</code>	Match rows using LIKE pattern
<code>join(\$table, \$a, \$op, \$b)</code>	INNER JOIN
<code>leftJoin(\$table, \$a, \$op, \$b)</code>	LEFT JOIN
<code>rightJoin(\$table, \$a, \$op, \$b)</code>	RIGHT JOIN
<code>crossJoin(\$table)</code>	CROSS JOIN
<code>groupBy(...\$cols)</code>	Group results by column(s)
<code>having(\$col, \$op, \$val)</code>	Filter grouped results
<code>orderBy(\$col, \$dir)</code>	Sort results (ASC by default)
<code>orderByDesc(\$col)</code>	Sort results descending
<code>limit(\$n)</code>	Limit number of results
<code>offset(\$n)</code>	Skip N rows
<code>paginate(\$page, \$perPage)</code>	Set limit/offset from page number
<code>get(\$excluding)</code>	Execute and return all rows
<code>first(\$excluding)</code>	Execute and return first row
<code>find(\$id, \$pk, \$excluding)</code>	Find by primary key (model only)
<code>all(\$excluding)</code>	Return all rows, no conditions (model only)
<code>count()</code>	Return total row count
<code>paginateResult(\$page, \$perPage)</code>	Return data + pagination metadata
<code>insert(\$data)</code>	Insert one row, return inserted ID
<code>insertMany(\$rows)</code>	Insert multiple rows, return affected count
<code>update(\$data)</code>	Update matching rows, return affected count
<code>delete()</code>	Delete matching rows, return affected count